

Python Fundamentos

Debugging

1



Programar no se trata solo de escribir código, sino de asegurarse de que este funcione correctamente y con un rendimiento aceptable. El proceso para lograrlo sigue un orden lógico:

- **Pruebas (Testing):** Verificar que el código se ejecute correctamente bajo condiciones controladas y los resultados sean los esperados.
- **Depuración (Debugging):** Identificar y corregir errores en el código. Una vez identificadas las causas, la corrección suele ser sencilla.
- **Optimización:** Mejorar el rendimiento del código, pero solo después de haber corregido los errores.

En este tema, nos centraremos en la depuración de código, una habilidad esencial para cualquier programador. Podemos utilizar desde utilizar simples declaraciones **print** hasta herramientas más avanzadas como **pdb** o **tracemalloc**.





Debido al ciclo de desarrollo rápido de Python, una de las formas más efectivas de depurar es modificar el código para que muestre información relevante en puntos clave. Python ofrece múltiples herramientas que permiten al código inspeccionar su propio estado y extraer información útil para la depuración. Entre estas herramientas, módulos como **traceback** son fundamentales.

Una vez que tienes la información relevante, el uso de **print** es la forma más directa de mostrarla. Sin embargo, en muchos casos, es más recomendable registrar la información de depuración en archivos, especialmente en programas que funcionan sin supervisión, como servidores. El módulo **logging** de Python es ideal para esta tarea, ya que facilita el registro de eventos importantes y errores.

Para una depuración más interactiva, Python también ofrece el módulo **pdb**, un depurador en modo texto que permite ejecutar el código paso a paso, inspeccionar variables y establecer puntos de ruptura. Si prefieres una interfaz gráfica, muchos **IDEs** ofrecen depuradores más avanzados.

Traceback

2



Cuando ocurre un error no manejado, Python genera automáticamente un **traceback**, describiendo el lugar exacto donde ocurrió el error y las funciones que se llamaron antes.

Los **tracebacks** son esenciales para identificar y entender los errores en Python. Un traceback muestra la secuencia de llamadas a funciones (call stack) que llevó a la excepción, proporcionando el archivo, el número de línea y el tipo de error. Por ejemplo:

```
>>> def spam():
...     bacon()
...
>>> def bacon():
...     raise Exception("Este es el error.")
...
>>> spam()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in spam
  File "<stdin>", line 5, in bacon
Exception: Este es el error.
```

Este ejemplo muestra cómo se encadenaron las llamadas a las funciones spam() y bacon(), hasta que ocurrió la excepción (3).



El módulo `traceback` permite manejar y personalizar la salida de los `tracebacks`, lo que es útil para capturar errores sin detener el programa abruptamente:

- **`traceback.print_exc()`**: Imprime el `traceback` de una excepción capturada.
- **`traceback.format_exc()`**: Devuelve el `traceback` como una cadena, útil para guardarlo en archivos de log.

```
>>> import traceback
>>>
>>> try:
...     1 / 0
... except Exception as e:
...     with open('error_log.txt', 'w') as f:
...         f.write(traceback.format_exc())
...     print("Se ha guardado el traceback en error_log.txt.")
...
Se ha guardado el traceback en error_log.txt.
```

Print

3



Al depurar un programa en Python, el uso de la función `print` junto con cadenas de formato (f-strings), puede ayudarte a identificar problemas rápidamente.

La función `print` genera una versión legible de cualquier valor que le pases. Por ejemplo, al imprimir una cadena de texto, se muestra el contenido sin las comillas.

```
>>> print('foo bar')
foo bar
```

Esto es equivalente a lo anterior:

```
>>> my_value = 'foo bar'
>>> print(str(my_value))
foo bar
>>> print('%s' % my_value)
foo bar
>>> print(f'{my_value}')
foo bar
>>> print(format(my_value))
foo bar
>>> print(my_value.__format__('s'))
foo bar
>>> print(my_value.__str__())
foo bar
```



Sin embargo, en algunas ocasiones, solo mostrar la versión legible de un valor no es suficiente, ya que no proporciona información sobre el tipo de dato o su composición interna.

En debugging, es común usar **repr** para asegurarte de que las diferencias de tipo se vean claramente. La función **repr** genera una representación que intenta ser más informativa, mostrando cómo es el objeto de manera que puedas recrearlo. Por ejemplo:

```
>>> print(repr(5))
5
>>> print(repr('5'))
'5'
```

```
>>> int_value = 5
>>> str_value = '5'
>>> print(f'{int_value!r} != {str_value!r}')
5 != '5'
```

Con f-strings podemos usar `!r`

```
>>> int_value = 5
>>> str_value = '5'
>>> print(f'{int_value!r} != {str_value!r}')
5 != '5'
```

pdb

4



Inspirado en el depurador gdb de C, pdb te permite establecer puntos de interrupción, inspeccionar variables y ejecutar código paso a paso. Esto es particularmente útil cuando necesitas entender el comportamiento del programa en tiempo real.

Para activar el depurador en cualquier punto de tu programa, puedes insertar el siguiente código:

```
import pdb; pdb.set_trace()
```

Este comando inicia la sesión interactiva de depuración, cambiando el prompt habitual de Python (>>>) al de depuración (Pdb). Desde aquí, puedes evaluar código, inspeccionar variables, y continuar con la ejecución.

```
def sumar(a, b):  
    result = a + b  
    return result  
  
def main():  
    breakpoint()  
    total = sumar(3, 5)  
    print(f'El total es: {total}')
```

main()

Un método más limpio a partir de Python 3.7 es usar la función `breakpoint()`, que es una función de conveniencia que llama a `pdb.set_trace()`:

Al ejecutar este código, el depurador se activa en el punto donde se encuentra el `breakpoint()`. Esto te permitirá inspeccionar el estado del programa en ese momento y utilizar los comandos de pdb para continuar o revisar el código.



Comandos básicos de pdb

Una vez en el prompt (Pdb), puedes usar una serie de comandos para controlar la depuración. Aquí están los más importantes:

Comando	Descripción
h o help	Muestra los comandos disponibles.
n o next	Ejecuta la siguiente línea de código sin entrar en funciones
s o step	Ejecuta la siguiente línea de código y entra en las funciones.
c o continue	Continúa la ejecución hasta el siguiente <i>breakpoint</i>
p variable	Imprime el valor de una variable.
l o list	Lista el código fuente alrededor de la línea actual.
q o quit	Salte de la sesión de depuración.
locals()	Muestra todas las variables locales en el contexto actual.

Ejemplo de ejecución línea por línea:

```
> /path/to/script.py(10)main()
-> total = sumar(3, 5)
(Pdb) n
> /path/to/script.py(11)main()
-> print(f'El total es: {total}')
(Pdb) p total
8
```

Inspección de variables

```
(Pdb) p total
8
(Pdb) locals()
{'a': 3, 'b': 5, 'result': 8}
```

Depuración en
IDEs

5



Además del depurador interactivo de Python pdb, entornos de desarrollo integrados (IDEs) como Spyder o PyCharm, ofrecen herramientas más visuales y fáciles de usar para depurar programas.

A diferencia de pdb, los depuradores de IDEs ofrecen una interfaz gráfica que permite visualizar el código, los puntos de interrupción y las variables de forma clara y organizada. También proporcionan botones dedicados para ejecutar el código línea por línea (step over), entrar en funciones (step into) o salir de funciones (step out) sin tener que recurrir a comandos de teclado, lo que hace que la navegación y el control del flujo de ejecución sea mucho más fácil.

Otra de las características avanzadas es la capacidad de realizar depuración remota mediante conexiones SSH. Esto es particularmente útil si tu aplicación está en un servidor o entorno remoto. IDEs como PyCharm permiten establecer una conexión SSH y depurar el código que se ejecuta en ese servidor remoto como si estuviera en tu máquina local.



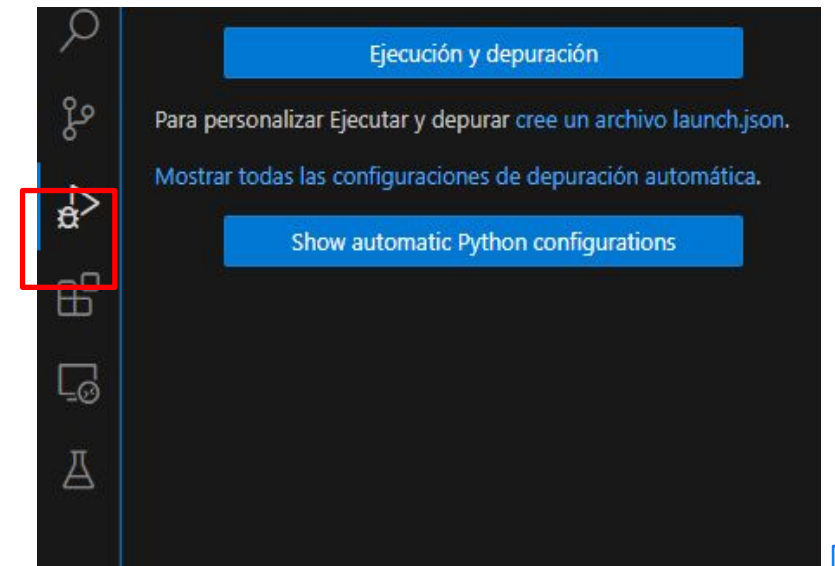
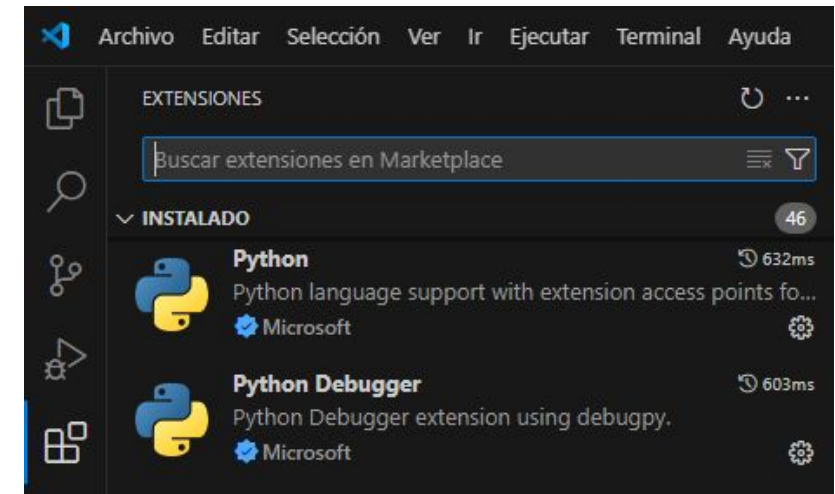


Visual Studio Code (VSCode), uno de los editores de código más populares, cuenta con un potente depurador que simplifica el proceso de depuración. Para configurar el Depurador en VSCode:

Asegúrate de que la extensión de Python esté instalada desde el Marketplace de VSCode.

Si nunca has configurado el depurador antes, deberás crear una configuración de depuración. Haz clic en el ícono de depuración en la barra lateral izquierda (Ctrl + Shift + D), y luego selecciona "Ejecución y depuración (Run and Debug)".

VSCode te ofrecerá crear una configuración predeterminada para Python (**launch.json**).

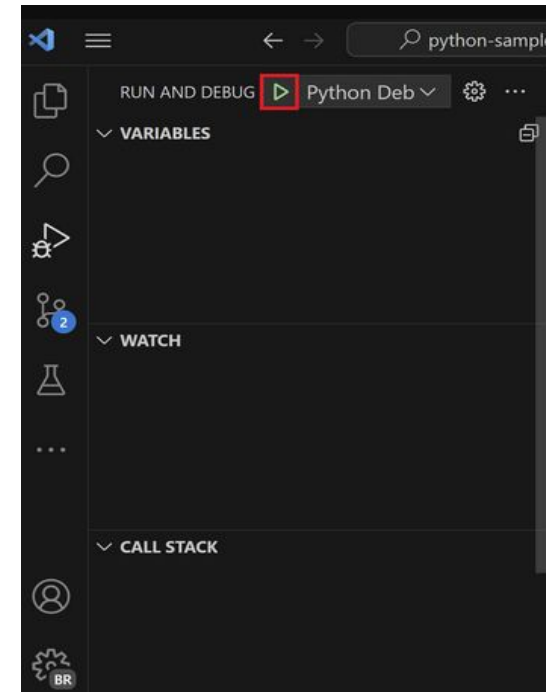




- Haz clic a la izquierda del número de línea en cualquier parte del código para establecer un breakpoint. El programa se detendrá en esa línea durante la ejecución.
- Para iniciar el depurador, presiona F5 o selecciona el botón de play en la barra superior de la sección de depuración. Esto iniciará tu programa y detendrá la ejecución en los puntos de interrupción que hayas definido.
- Cuando el código se detiene en un punto de interrupción, puedes inspeccionar las variables locales y globales en el panel de Variables de la interfaz de depuración. También puedes pasar el cursor sobre las variables en el código para ver sus valores actuales.

```
suma.py > sumar
1 def sumar(a, b):
2     result = a + b
3     return result
4
5 def main():
6     breakpoint()
7     total = sumar(3, 5)
8     print(f'El total es: {total}')
9
10 main()
```

A red box highlights the left margin of the code editor, where a red dot indicates a breakpoint has been set on line 7.





También puedes utilizar la consola integrada para ejecutar comandos de Python en el entorno actual del depurador.

VSCoide ofrece botones para continuar la ejecución, avanzar al siguiente paso o entrar en una función:

- Step Over (F10): Ejecuta la línea actual y avanza a la siguiente, sin entrar en las funciones llamadas.
- Step Into (F11): Entra en la primera función que se llama en la línea actual.
- Step Out (Shift+F11): Sale de la función actual y regresa a la llamada anterior.

The screenshot shows the Visual Studio Code interface. The top editor pane displays the file 'suma.py' with the following code:

```
1 def sumar(a, b):  
2     result = a + b  
3     return result  
4  
5 def main():  
6     total = sumar(3, 5)  
7     print(f'El total es: {total}')  
8  
9 main()
```

The bottom pane shows the 'CONSOLA DE DEPURACIÓN' (Debug Console) with the following output:

```
→ total  
8  
→ dir(sumar)  
['__annotations__', '__builtins__', '__call__', '__class__', '__closure__', '__code__', '__default__',  
> ' ', '__get__', '__getattr__', '__getstate__', '__globals__', '__gt__', '__hash__', '__init__',  
  'ne', '__new__', '__qualname__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__',  
→ sumar.__sizeof__()  
144
```

¡Gracias!